Scalable Network I/O: From select to io_uring

CS331 Computer Networks | Team T016 | Project ID 13

Kale Parth Hemant *24110242*

Devvrat Hans *23110094*

Patil Nachiket Kiran *24110250*

Niraj Kumar *24110222*

Chirag Agarwalla *24110093*

Nityanand Karmali *24110226*

I. Introduction

Chat servers, file servers, video relays and key-value stores all keep many network connections open at once. Giving each connection its own thread wastes memory and causes scheduler overhead, so most servers use an *event loop*: one thread asks the kernel which sockets are ready, serves those, and asks again. How the thread asks is the I/O multiplexing mechanism, and its cost is paid on every request.

Linux has four generations of this mechanism. `select` (1983) and `poll` (1986) pass the full list of interesting descriptors on every call, so the kernel scans all of them. `epoll` (2002) registers interest once and returns only ready descriptors. `io_uring` (2019) changes the model: the application submits whole operations (accept, receive, send) to a ring in shared memory and later reads their completions, so one system call can carry many operations, or none at all.

The common wisdom is "`epoll` scales, `select` does not, `io_uring` is fastest". Published measurements are more mixed [3], [7], and the answer depends on the workload. This project measures *when* each mechanism is better and explains *why* in operating-system terms.

Contributions. (1) A plugin framework that runs six applications, over TCP and UDP, on four interchangeable engines that share all other code, so the engines are the only variable. (2) A measurement method with exact in-server system-call counters (checked against `strace`), per-core interrupt and softirq accounting, run-queue wait, and closed-loop and paced load that avoids coordinated omission. (3) 18 experiment suites (1140 valid runs) that vary connections, idle descriptors, message size, transfer direction, transport, offered load, buffer size and thread count, plus the extensions the TA asked for: level- versus edge-triggered `epoll`, `io_uring` SQPOLL, and multithreaded servers. (4) An explanation of each result in terms of system calls, scanning, wakeups, interrupts, CPU and memory.

In short, no mechanism wins everywhere. With every connection busy, `select` and `poll` match or beat default `epoll`, which pays two `epoll_ctl` calls per request. With idle connections, `select` and `poll` lose 78% of their throughput between 10 and 1000 descriptors while `epoll` and `io_uring` stay flat. `io_uring` gives 6–15% more throughput when batching is possible and nothing when it is not, and its CPU use follows the offered load; only the SQPOLL mode keeps a core busy at every load.

II. Problem Statement

Problem. Given a single-threaded server on Linux, determine how the choice of I/O multiplexing mechanism affects throughput, latency, CPU cost and kernel activity, and how that answer changes with the application and the load.

Inputs (independent variables): mechanism (`select`, `poll`, `epoll` in three modes, `io_uring` in three modes); application (noop, echo, upload, download, stream, memory); transport (TCP, UDP); number of connections (1–1000) and how many are idle; message size (1 B–1 MiB); arrival model (closed loop, or a fixed paced rate of 1k–40k req/s); server buffer size (4–64 KiB); number of event-loop threads (1, 2, 4).

Outputs (measured): verified requests per second, payload MiB/s, latency p50/p99/p99.9, server CPU (user and kernel), CPU time per request, system calls per message by type, voluntary context switches (wakeups), network softirqs and hardware interrupts on the server core, run-queue wait, and peak resident memory.

Assumptions and constraints. One machine, loopback networking (to isolate the software cost of each mechanism from a network card), non-blocking sockets, and an unmodified Ubuntu kernel. The client must never be the bottleneck, and every run must pass a correctness check.

In scope: event-loop servers with one or more independent loops. **Out of scope:** thread-per-connection servers, real NICs and RDMA, kernel bypass (DPDK), storage I/O, and `io_uring` multishot receive (not available in liburing 2.1).

Gap. Earlier studies compared `select`, `poll` and `epoll` on one web workload [3], or `io_uring` on storage [6]. Our first version also compared the engines on a single echo benchmark, which cannot show that the best mechanism depends on the application. A fair comparison needs the same application code on every engine, several application types, and an explanation from the OS side, which is what this project provides.

III. Project Objectives

1) Build the framework first: a shared core (sockets, request state machine, UDP, threads, signals, statistics) and a plugin interface, so any application runs on any engine over TCP or UDP.

2) Implement four engines that contain *only* their event loop: `select`, `poll`, `epoll` (level-triggered, edge-triggered, level-triggered with immediate reply) and `io_uring` (plain, SQPOLL, SQPOLL with busy completion polling).
3) Provide a no-op baseline in which the loop does no useful work, giving each engine's top request frequency.
4) Measure latency-sensitive, read-heavy, write-heavy, streaming and memory-heavy applications, over TCP and UDP, with 1 to 1000 descriptors.
5) Answer the TA's question on `io_uring` CPU use by measuring CPU against a controlled offered load.
6) Record wakeups, interrupts and softirqs, explain every difference from the OS point of view, and check every measurement for correctness.

IV. Background and Related Work

Readiness versus completion. `select`, `poll` and `epoll` report *readiness*: the kernel says a socket can be read or written without blocking, and the program then calls `recv` or `send`. `io_uring` reports *completion*: the program hands the kernel a `recv` with a buffer, and later reads a completion entry stating how many bytes arrived.

Cost per call. `select` copies three bitmaps of up to 1024 bits (`FD_SETSIZE`) into the kernel; the kernel calls each socket's `poll` method, adds the thread to each socket's wait queue, sleeps, and on wakeup scans every descriptor again because it does not know which one woke it. `poll` replaces the bitmaps with an array of `struct pollfd`, which removes the 1024 limit but keeps the $O(N)$ scan. `epoll` keeps an interest set in the kernel: sockets that become ready put themselves on a ready list through a callback, so `epoll_wait` costs $O(\text{ready})$. Changing interest (for example, from reading to writing) costs an `epoll_ctl` system call. In level-triggered (LT) mode a ready socket is reported again on every wait; in edge-triggered (ET) mode only when its state changes, so the program must read or write until `EAGAIN`.

`io_uring` shares a submission queue (SQ) and a completion queue (CQ) with the kernel. One `io_uring_enter` call can submit many operations and wait for completions; if completions are already in the CQ, no system call is needed at all. With SQPOLL, a kernel thread polls the SQ, so the application does not need to make system calls to submit either, at the cost of a thread that spins while there is work and sleeps after an idle timeout.

Related work. Banga, Mogul and Druschel showed that `select` scales poorly as descriptors grow and proposed registering interest once, the idea behind `epoll` [1]. Provos and Lever made `poll` cheaper with `/dev/poll` and compared it with POSIX real-time signals [2]. Gammo et al. found `select` and `poll` comparable to `epoll` when there are no idle connections, and attributed part of `epoll`'s cost to `epoll_ctl` [3]. FlexSC showed the benefit of batching system calls through shared memory [4], the idea `io_uring` builds on [8]. Schroeder et al. showed that closed and open load generators can lead to different conclusions [5], which motivated our paced mode. Didona et al. studied `io_uring` polling modes for storage [6], and a recent DBMS study found that replacing `epoll` with `io_uring` gave about 1.1× for network traffic, with SQPOLL worthwhile only when a dedicated core is justified [7]. These papers predict that (a) with all connections active the readiness mechanisms are close, (b) with idle connections `select`/`poll` degrade and `epoll` does not, and (c) `io_uring` gives a modest gain. We test these predictions on six applications and over UDP.

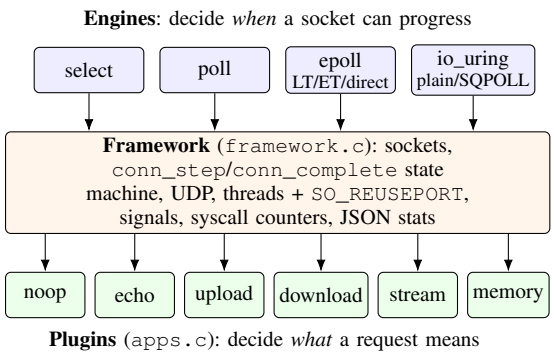


Fig. 1. Three layers. Every engine implements one function, `engine_run`; every plugin implements the `app_ops_t` callbacks. The framework and plugins are compiled once and linked into all four servers. The C load generator connects over loopback.

V. Architecture and Design

Fig. 1 shows the three layers. An engine never looks at an application and an application never makes a socket call, so any plugin runs on any engine over TCP or UDP.

Plugin contract. A plugin provides `validate(length)`, `body_size` and `response_size` (bytes in and out), `on_body` (consume received bytes), `fill` (produce reply bytes), `on_open`/`on_close` (per-client state) and `on_datagram` (the same application as one UDP datagram). The six plugins are: **noop**, an 8-byte header answered with one byte (the baseline where the loop does no useful work); **echo**, a latency-sensitive chat-like exchange; **upload**, a read-heavy transfer in which every byte is verified; **download**, a write-heavy transfer of a known byte pattern; **stream**, 64 KiB frames paced by the client like video; and **memory**, in which every client owns a resident 256 KiB buffer that each request writes into.

Wire format. A TCP request is an 8-byte big-endian length followed by the body; the reply length is set by the plugin. A UDP datagram carries an 8-byte sequence number, the requested reply length and the body, since UDP keeps message boundaries.

Shared request state machine. Each client is a `conn_t` with a stage (`HEADER`, `RECEIVE`, `SEND`), progress counters `done`/`want`, and a buffer (4 KiB by default). `conn_step` performs exactly one `recv` or `send`, whichever the stage needs, and passes the result to `conn_complete`, which advances the state. `io_uring` calls `conn_complete` directly

with the result from a completion entry, so all four engines share the same state machine and partial reads and writes are handled once. A client never has a read and a write in flight together, so one buffer per client is safe.

Key design decisions. *(a) Shared code instead of one copy per engine.* In our first version each server had its own copy of the application code; GCC compiled a per-byte check differently in each, and that alone made two engines look 60% faster on upload. Compiling the framework and plugins once removed this bias. *(b) Counting system calls inside the server.* `strace` slows a server several times and changes its batching, so the servers count every I/O system call themselves; we validated the counters against `strace -c`. *(c) Loopback.* It removes NIC and wire effects so the measured differences are the mechanisms' own. The trade-off is that the receiver's network processing (the NET_RX softirq) runs on the sending CPU, which we account for explicitly. *(d) Independent loops for threads.* Each thread has its own listening socket with `SO_REUSEPORT`, so the kernel spreads connections and no lock is ever taken. A shared-queue design would have measured lock contention instead of the mechanisms. *(e) Signals.* `SIGINT`/`SIGTERM` handlers only set a flag (handlers may call only async-signal-safe functions); waits are not restarted (`SA_RESTART` unset) and time out every 100 ms, so the loop notices the flag. `SIGPIPE` is ignored and sends use `MSG_NOSIGNAL`, so a closed client gives `EPIPE` instead of killing the server.

VI. Implementation

The servers, framework and load generator are written in C11 (about 2,500 lines) and built with `gcc -O2 -Wall -Wextra -Werror`; `io_uring` uses liburing 2.1. The experiment runner and analysis are in Python 3. The code is at https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO.

select keeps a table of clients indexed by descriptor, rebuilds the read/write sets before each call (the kernel overwrites them), and walks descriptors up to the highest one. A descriptor $\geq$ `FD_SETSIZE` cannot be represented and is rejected cleanly. **poll** keeps a parallel `pollfd[]`/`clients[]` array with the listener in slot 0 and rebuilds interest each iteration. **epoll** registers each client once; in LT mode it switches interest between `EPOLLIN` and `EPOLLOUT` with `EPOLL_CTL_MOD` (two `epoll_ctl` calls per request); *LT-direct* tries the reply immediately and arms `EPOLLOUT` only if the send would block; ET registers `EPOLLIN|EPOLLOUT|EPOLLET` once and reads or writes until `EAGAIN`. Algorithm 1 shows the **io_uring** loop.

Load generator. `loadgen` is a multithreaded C client using `epoll`. In *closed-loop* mode each connection sends its next request as soon as the reply arrives (maximum throughput). In *paced* mode requests follow a fixed schedule and latency is measured from the scheduled send time, so a slow server cannot hide its delay (no coordinated omission [5]); threads are staggered and sleep until 50 µs before each send, then spin. Latency goes into a log-linear histogram and every reply byte is verified.

Algorithm 1 io_uring event loop (one thread)

```
set up ring: SQ depth 256, CQ sized for all clients (IORING_SETUP_CQSIZE); optional SQPOLL pinned to its own core
queue one accept
while not stop_requested do
    submit queued SQEs                ▷ one io_uring_enter for all
    wait for ≥1 CQE (skip syscall if CQ non-empty; 100 ms timeout)
    for each available CQE do
        if accept then add client; queue recv; re-queue accept
        else r ← conn_complete(client, res)
            if r ≠ 0 close client else queue next recv or send
        end if
    end for
end while
cancel all in-flight operations, drain CQ, close clients
```

Runner and validation. `experiments.py` defines the 18 suites, pins processes to CPUs, starts each server and waits for `READY`, snapshots `/proc` (process CPU, context switches, run-queue wait from `schedstat`, per-core interrupts and softirqs) at the start and end of the measured window, stops the server with `SIGTERM` and reads its JSON statistics. A run is *valid* only if the server's and client's message counts agree, every byte count matches the protocol, all connections were accepted and there were no errors; invalid runs are kept on disk but never plotted.

Challenges solved. (1) The compiler-induced 60% bias described above. (2) With 1000 clients `io_uring`'s default 512-entry CQ overflowed and the kernel fell back to a slow path; the CQ is now sized for all clients. (3) An `io_uring` stall when the SQ filled, fixed by submitting before queuing more. (4) Paced client threads initially sent in bursts; staggering their start times fixed it. (5) UDP needed 8 MiB socket buffers and a configurable number of outstanding receives (`--udp-depth`) to avoid drops. (6) Latency spikes were traced (with 10 ms sampling) to other tasks briefly using the server's CPU, not to any engine.

VII. Testbed Setup

Hardware. ASUS ROG Flow X13 (GV301RC): AMD Ryzen 9 6900HS, 8 cores / 16 threads, 14 GiB usable RAM, on mains power. **Software.** Ubuntu 22.04.5 LTS, Linux 6.8.0-138-generic, gcc 11.4.0, liburing 2.1, Python 3.10. **Configuration.** CPU governor `performance`; `net.core.rmem_max` and `wmem_max` = 8 MiB. The runner refuses to start if these are not set, and records them. **Topology.** Client and server on the same machine over the loopback interface (127.0.0.1). **Placement.** The server's event loop is pinned to CPU 0 (and CPUs 0, 2, 4, 6 with four threads); the SQPOLL kernel thread to CPU 2; the load generator's up to 8 threads to the remaining physical cores and their SMT siblings, never a server CPU; the controller to CPU 15. **Runs.** Each run has a 2 s warm-up and a 10 s

TABLE I
CLOSED LOOP, 100 CONNECTIONS, ALL ACTIVE (MEDIAN OF 5)

	noop (1 B reply)		echo 64 B			
Engine	k req/s	sc/msg	k req/s	CPU/req	sc/msg	p99
select	185.0	2.02	165.1	6.06 µs	3.03	635 µs
poll	185.7	2.02	166.1	6.02 µs	3.03	758 µs
epoll LT	158.0	4.02	140.7	7.10 µs	5.03	799 µs
io_uring	**193.3**	0.02	**176.3**	**5.67 µs**	0.03	594 µs

measured window. Each of the 228 settings was repeated 5 times, and all 1140 runs were shuffled together (seed 2026) so slow drift cannot favour one engine. All 1140 runs were valid. Across the 228 settings the median spread (max−min over median) of throughput was 1.3%. No load-generator thread exceeded 73% of a CPU in any run, so the client was never the bottleneck, and no UDP datagram was dropped or timed out. Tables show medians; figures show the median and the min–max band.

Correctness evidence. 159 automated tests on 15 server configurations (all engines and modes, 2 threads, 64 KiB buffers, UDP depth 1) covering echo, byte-verified upload and download, error handling and shutdown, client limits, 1000 connections, invalid requests, resets during a send, signals and bad options, every plugin and UDP: 159/159 passed. The in-server counters matched `strace -c` for six variants (exactly for `recv`, `send`, `accept` and `epoll_ctl`; within 0.03% for `io_uring` waits). The same paced load from our C client and an independent Python client gave the same rate and the same I/O calls per message.

VIII. EVALUATION

Unless stated otherwise, "syscalls/msg" counts I/O system calls per request (`recv`, `send`, waits, `epoll_ctl`, `io_uring_enter`), and "CPU/req" is server CPU time per request. In closed-loop runs the server is the bottleneck, so its CPU is always 100% and the useful comparisons are throughput and CPU per request.

A. Baseline and all-active connections

Table I shows the baseline (noop) and the echo application with every connection busy. Three effects explain it.

Batching amortises the scan. With 100 busy clients, almost all are ready on each wakeup, so one `select` or `poll` call serves about 33 requests (0.03 waits per message). The $O(N)$ scan is spread over many requests and costs little per request.

epoll pays for changing interest. Default (LT) `epoll` needs 2 `epoll_ctl` calls per request (switch to `EPOLLOUT` to send, then back to `EPOLLIN`), giving 5.03 system calls per message instead of 3.03. This costs about 1 µs per request and 15% of throughput, exactly the effect Gammo et al. attributed to `epoll_ctl` [3]. The LT-direct mode, which replies immediately, removes it (161.7 k req/s, 3.02 syscalls/msg; Table III).

io_uring batches across clients. With many completions per `io_uring_enter`, `io_uring` needs only 0.03 kernel

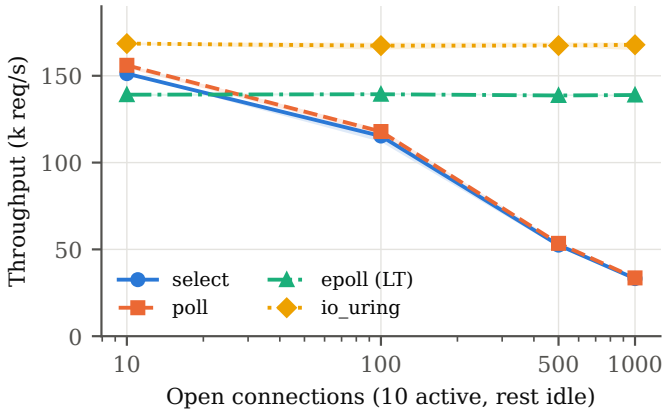


Fig. 2. Echo throughput with 10 active clients and an increasing number of idle connections (closed loop, median and min–max of 5 runs).

entries per message. It is 6–7% faster than `select`/`poll` and 4% faster on the noop baseline. The gain is modest because each `recv`/`send` still runs the same TCP code inside the kernel; `io_uring` removes the system-call entries, not the network stack work, which is consistent with the 1.1× reported in [7].

With a single connection there is nothing to batch: all four engines reach 64–67 k req/s at 58–62% CPU, because the loop sleeps and is woken (one voluntary context switch) for every request. With 1000 connections, `select`, `poll` and `io_uring` stay at 163–165 k req/s and LT `epoll` at 135 k req/s.

B. Idle connections: the $O(N)$ scan

Fig. 2 keeps 10 clients busy and adds idle connections. `select` falls from 151.2 to 33.2 k req/s and `poll` from 156.0 to 33.6 k req/s at 1000 descriptors, a 78% loss, and their CPU per request rises from about 6.5 to 30 µs. The system calls per message stay at 3.3, so the extra cost is *inside* each call: the kernel copies the interest list and calls every socket's `poll` method twice, ready or not. `epoll` (139 k req/s) and `io_uring` (168 k req/s) are flat, because their cost depends on ready events, not registered descriptors. This confirms Banga et al. [1]. The crossover is small: already at 100 descriptors, with 90 of them idle, `select` and `poll` fall below `epoll`.

C. Does CPU follow load? (TA's io_uring question)

In closed-loop runs every engine shows about 100% CPU because the client sends as fast as possible; this is what made `io_uring` look like "90% CPU at any load". Fig. 3 instead fixes the offered rate. **Plain `io_uring`'s CPU follows load**, from 3.4% at 1k req/s to 40.9% at 40k, the same as `epoll` (2.8% to 41.1%). It sleeps in `io_uring_enter` when there is nothing to do, with one voluntary context switch per request. Only **SQPOLL** stays above one core at every load (101% at 1k req/s, 129% at 40k), because its kernel thread polls the SQ and sleeps only after 100 ms with no work, which never happens at these rates. With busy completion polling as well,

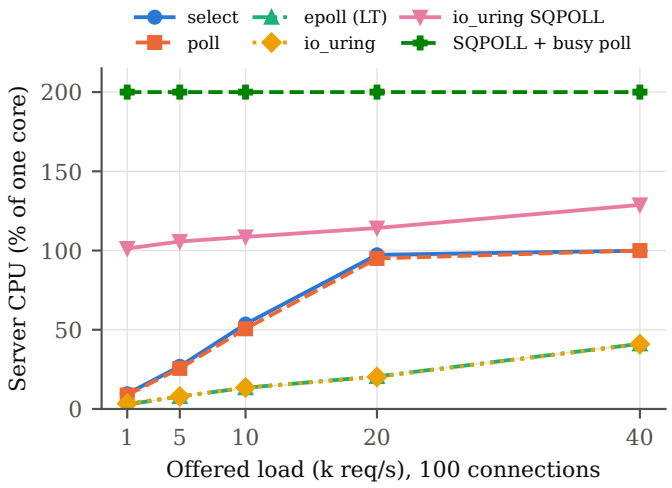


Fig. 3. Server CPU against a paced offered load (echo 64 B, 100 connections). Plain io_uring and epoll overlap.

TABLE II
BULK TRANSFER, 1 MiB REQUESTS, 10 CONNECTIONS, 4 KiB BUFFER

	Upload (read-heavy)			Download (write-heavy)		
Engine	MiB/s	CPU/MiB	sc/MiB	MiB/s	CPU/MiB	sc/MiB
select	2681	373 µs	292	587	1705 µs	283
poll	2849	351 µs	291	612	1634 µs	283
epoll LT	2764	362 µs	293	592	1688 µs	285
io_uring	**3097**	**323 µs**	27	609	1640 µs	26

it uses exactly 200% (two cores) at every load, for the lowest latency of all (p50 14.8 µs at 40k req/s, against 17.0 µs for plain `io_uring`).

The figure also shows a second cost of scanning. Under light paced load each wakeup finds about one ready socket, so `select` and `poll` scan all 100 descriptors *per request*: they cost 48–54 µs per request against 10–16 µs for `epoll` and `io_uring`, saturate one core at about 20k req/s, and have three times the median latency at 40k req/s (54 µs against 18 µs). In closed loop the same scan was amortised over about 33 requests (Table I), so *how bursty the load is* decides whether the scan matters.

D. Bulk transfer: upload, download and buffer size

Table II compares the two directions. On upload, `io_uring` is 9–15% faster and needs 27 kernel entries per MiB instead of about 292, but the CPU per MiB falls by only 8–13%: almost all of it is kernel time copying data, which every engine pays. On download all engines are within 5%. Download costs about 4.6× more CPU per MiB than upload because on loopback the *sender's* CPU also runs the receiver's network processing: we counted 256 NET_RX softirqs per 1 MiB request on the server core, one per 4 KiB segment sent. This is a property of loopback, and it is why we account softirqs separately from the process's own CPU time.

TABLE III
EPOLL MODES (CLOSED LOOP, MEDIAN OF 5)

	echo 64 B, 100 conns			download, 10 conns	
Mode	k req/s	sc/msg	ctl/msg	MiB/s	waits/MiB
LT (default)	141.2	5.03	2.00	602	25.7
LT-direct	**161.7**	3.02	0.00	605	25.6
ET	154.5	4.01	0.00	**685**	0.1

TABLE IV
IO_URING MODES, ECHO 64 B, CLOSED LOOP

	10 connections			500 connections		
Mode	k req/s	CPU%	ent./msg	k req/s	CPU%	ent./msg
plain	**168.2**	100	0.30	**169.6**	100	0.01
SQPOLL	145.9	112	0.30	160.5	200	0.00
SQPOLL+busy	167.5	200	0.00	161.8	200	0.00

The server buffer size matters far more than the engine. With 64 KiB instead of 4 KiB buffers, download rises from 592 to 4519 MiB/s (7.6×, `select`) and upload from 2679 to 10015 MiB/s, because each system call moves 16 times more data (`select`: 283 to 19 syscalls per MiB). At 64 KiB the engines are within 5% of each other (download 4330–4519, upload 10015–10254 MiB/s).

E. Level- versus edge-triggered epoll

Table III answers the TA's LT/ET question. For small echo messages the cost that matters is `epoll_ctl`: both LT-direct and ET avoid it, and LT-direct is fastest. ET pays one extra `recv` per request, the one that returns `EAGAIN`, which ET requires to be sure the socket is drained. For downloads ET wins by 14%: one edge notification is followed by sends until the socket buffer is full, so it needs 0.1 waits per MiB instead of 25.7. LT reports the socket again after every 4 KiB send. The best mode therefore depends on the application: LT-direct for request–response, ET for bulk writes.

F. io_uring SQPOLL under full load

Table IV shows that SQPOLL does not raise throughput on this machine. Plain `io_uring` already needs very few kernel entries per message under load (0.01 at 500 connections), so there is little left for SQPOLL to remove, while the extra thread doubles the CPU and adds hand-off latency between the two cores. At 10 connections, plain SQPOLL is 13% slower because the application still sleeps in the kernel waiting for completions (0.30 entries per message) and must also wake the SQ thread. SQPOLL is worth its extra core only when the lowest latency matters more than CPU (Fig. 3), as [7] concluded.

Paced SQPOLL also had a heavier tail: its median p99 at 20k–40k req/s was 1.1–1.2 ms, against 21–29 µs for the other `io_uring` modes. Across these runs, p99 correlates at 0.96 with the time our application thread spent waiting in the run queue on CPU 0, which is also used by other system tasks.

TABLE V
STREAM, MEMORY AND UDP (MEDIAN OF 5)

	select	poll	epoll	io_uring
Stream: 100 conns × 30 frames/s of 64 KiB (187.5 MiB/s)				
Server CPU %	96.7	94.4	30.2	**29.4**
p50 latency (µs)	338	326	109	**107**
Memory: 500 conns, 256 KiB per client, 4 KiB writes				
k req/s	148.4	149.4	125.9	**149.9**
Peak RSS (MiB)	130.5	130.6	130.5	130.8
UDP echo, 100 flows, one socket (k req/s); io_uring at depth 1 / 32				
64 B	**242.0**	241.6	234.3	199.2 / 237.2
1400 B	**232.9**	232.8	226.4	192.2 / 225.1
4096 B	175.9	175.3	173.0	176.9 / **201.1**

When that CPU is briefly taken, the SQ thread on another core keeps polling but the completions wait. The tail is scheduler interference on a shared core, not `io_uring` itself, and our 10 ms spike sampling found the same cause for the rare spikes in other engines.

G. Streaming, memory-heavy service and UDP

Streaming (Table V) is paced, so it behaves like the rate sweep: each 64 KiB frame is sent in 16 pieces of 4 KiB, about 17 waits per frame, and each `select`/`poll` wait scans all 100 descriptors. They therefore need 3.2× the CPU of `epoll`/`io_uring` (322 against 100 µs per frame) for the same data, and their median frame latency is three times higher.

Memory-heavy service. Each of 500 clients holds a resident 256 KiB buffer (130 MiB peak RSS for every engine, against 1.8 MiB for echo). The engine ranking is the same as for echo: `io_uring`, `poll` and `select` at 148–150 k req/s and LT `epoll` 15% lower because of `epoll_ctl`. Memory per client is a property of the application, not of the multiplexing mechanism: with echo, every engine's server stays at 1.8–2.5 MiB.

UDP. A UDP server uses one socket for all clients, so there is no $O(N)$ scan and `select` and `poll` are as fast as `epoll`. `io_uring` with one outstanding receive (depth 1) is 18% *slower* for small datagrams: each datagram needs two kernel entries (submit the receive, then the send) and nothing is batched. With 32 receives outstanding it needs 0.06 entries per datagram and matches the readiness engines; for 4096 B datagrams it is 14% faster. As with TCP, `io_uring` helps only when it can batch.

H. Multiple event loops

Fig. 4 shows near-linear scaling for every engine, because the loops share nothing: four loops give 3.65× (`select`), 3.70× (`poll`), 3.87× (`epoll`) and 3.98× (`io_uring`). `io_uring` reaches 648 k req/s with four loops. The ranking with one loop is preserved, so the mechanism's per-request cost still matters after scaling out.

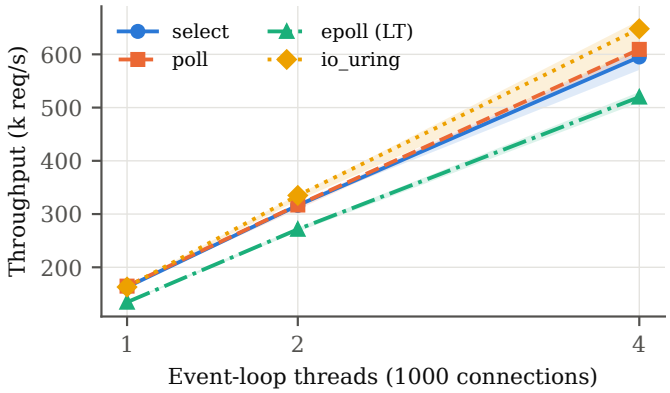


Fig. 4. Echo 64 B with 1000 connections spread over 1, 2 and 4 independent event loops (`SO_REUSEPORT`), one per physical core.

TABLE VI
WHICH MECHANISM TO USE

Situation	Best choice and reason
All connections busy, small messages	`io_uring` (+6–7%); `select`/`poll` next; LT `epoll` worst (2 `epoll_ctl`/req)
Many idle connections	`epoll` or `io_uring`; `select`/`poll` lose 78% at 1000 fds
Light or paced load	`epoll`/`io_uring`; `select`/`poll` scan all fds per request
Bulk transfer	Buffer size dominates (up to 7.6×); engines within 5% at 64 KiB
Bulk writes with epoll	ET (+14%)
UDP (one socket)	Any readiness engine; `io_uring` only with deep queues
Lowest latency, spare cores	SQPOLL + busy polling (2 cores at any load)

I. Interrupts and wakeups

On loopback no device interrupts reached the server core in any run; the network work appears as softirqs instead. Each echo request causes one NET_RX softirq on the server core (for the reply it sends), and each 1 MiB download causes 256. In closed loop no engine sleeps (0 voluntary context switches per message), because work is always waiting. In paced mode every engine that is not saturated sleeps and is woken once per request; busy-polling SQPOLL never sleeps.

J. Summary of findings

Table VI summarises the results. The literature's predictions held: with all connections active the readiness mechanisms are close [3]; with idle connections `select` and `poll` degrade and `epoll` does not [1]; `io_uring`'s gain is modest [7]. We also found that under light load the scan cost appears per request, that `epoll`'s default LT mode is held back by `epoll_ctl`, and that UDP removes the scanning difference.

IX. Summary

We asked which Linux I/O multiplexing mechanism a single-threaded server should use, and found that the answer depends on the application and the load. Using one framework in which four engines share all application code, we measured six applications over TCP and UDP in 1140 validated runs and explained each result by system calls, scanning, wakeups, softirqs and CPU. `select` and `poll` are competitive when every connection is busy, but their per-call scan of all descriptors costs up to 78% of throughput with idle connections and dominates under light load. `epoll` is flat in the number of descriptors but its default level-triggered use pays two `epoll_ctl` calls per request, which the LT-direct and ET modes remove. `io_uring` is fastest wherever operations can be batched (6–15%), gains nothing without batching, and its CPU follows load; only SQPOLL trades a permanently busy core for latency.

Limitations. All results are on loopback, where receive processing runs on the sender's CPU, and on one laptop whose CPU 0 is shared with system tasks; upload and download use memory patterns instead of files; `io_uring` was limited to liburing 2.1 (no multishot receive, no registered buffers). **Future work.** Repeat on two machines with a real NIC and interrupt affinity; add multishot receive, registered buffers and zero-copy send; test thread-per-connection and shared-queue multithreaded designs; and run real file I/O.

X. Reproducibility

Repository: https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO. The engines are in `code/select`, `code/poll`, `code/epoll` and `code/io_uring`; the framework, plugins, load generator and scripts in `code/benchmark`.

Requirements: Linux $\geq$ 5.15 with `io_uring` enabled, gcc, make, `liburing-dev`, Python 3, `strace`, `iproute2`.

Steps:

1) Prepare the machine (mains power, Wi-Fi off):

```
sudo apt install build-essential \
    liburing-dev strace iproute2 python3
sudo cpupower frequency-set -g performance
sudo sysctl -w net.core.rmem_max=8388608
sudo sysctl -w net.core.wmem_max=8388608
```

2) Build and run everything: `cd code/benchmark && ./run_all.sh`. This checks the environment, builds with `make`, runs the correctness tests, the counter check against `strace`, the client cross-check, all benchmark suites (about 5 hours), the analysis and the latency-spike diagnosis.

3) Individual parts:

```
python3 correctness.py OUT
python3 experiments.py run \
    --suites idle-scaling --out OUT
python3 experiments.py analyze \
    --input OUT --out REPORT
```

4) Results: `results/final/report/report.md` (tables), `aggregates.csv` (all medians, min and max) and one folder per run with the exact server and client command lines and their JSON outputs.

References

[1] G. Banga, J. C. Mogul, and P. Druschel, "A scalable and explicit event delivery mechanism for UNIX," in *Proc. USENIX Annu. Tech. Conf.*, Monterey, CA, USA, 1999, pp. 253–266.

[2] N. Provos and C. Lever, "Scalable network I/O in Linux," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, San Diego, CA, USA, 2000.

[3] L. Gammo, T. Brecht, A. Shukla, and D. Pariag, "Comparing and evaluating epoll, select, and poll event mechanisms," in *Proc. Ottawa Linux Symp.*, Ottawa, Canada, 2004, pp. 215–226.

[4] L. Soares and M. Stumm, "FlexSC: Flexible system call scheduling with exception-less system calls," in *Proc. USENIX OSDI*, Vancouver, Canada, 2010.

[5] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. USENIX NSDI*, San Jose, CA, USA, 2006, pp. 239–252.

[6] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, "Understanding modern storage APIs: A systematic study of libaio, SPDK, and io_uring," in *Proc. ACM SYSTOR*, Haifa, Israel, 2022, pp. 120–127, doi: 10.1145/3534056.3534945.

[7] "io_uring for high-performance DBMSs: When and how to use it," arXiv:2512.04859, 2025.

[8] J. Axboe, "Efficient IO with io_uring," 2019. [Online]. Available: https://kernel.dk/io_uring.pdf

APPENDIX A: TEAM WORK DISTRIBUTION AND MEMBER RATING

A.1 Work Distribution

- Ratings are agreed by the whole team and are unique: no two members received the same rating (no ties).
- Scale: 1–10 (10 = highest contribution). With six members, six different values are used (10 to 5).

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Kale Parth Hemant	24110242	`select` engine (`select_server.c`): descriptor table, read/write set construction, `FD_SETSIZE` guard, TCP and UDP loops.	9
2	Devvrat Hans	23110094	`poll` engine (`poll_server.c`): parallel `pollfd`/client arrays, per-iteration interest rebuild, TCP and UDP loops.	6
3	Patil Nachiket Kiran	24110250	`epoll` engine (`epoll_server.c`) in level-triggered, edge-triggered and LT-direct modes; ran all experiments on his laptop (the testbed).	7
4	Niraj Kumar	24110222	`io_uring` engine (`io_uring_server.c`): plain, SQPOLL and busy-poll modes, completion-queue sizing, UDP receive depth, shutdown and cancellation.	5
5	Chirag Agarwalla	24110093	Benchmarking: shared framework and plugins, C load generator, experiment runner and analysis, correctness tests, counter and client cross-checks.	10
6	Nityanand Karmali	24110226	Report, presentation slides and literature review.	8

APPENDIX B: AI USAGE DETAILS

B.1 Prompts

The links to the prompts and chat logs for the most significant uses listed in the table are given below it.

#	AI Tool	Purpose
1	Claude, Gemini (Kale Parth Hemant)	Understanding the project and the four mechanisms; designing and implementing the `select` server.
2	Freebuff / Codebuff with DeepSeek V4 Flash (Devvrat Hans)	Understanding how `poll` differs from `select` inside the kernel; generating the `poll` server from his specification; adding comments; writing the component README.
3	Claude (Patil Nachiket Kiran)	Evolving his first `epoll` echo server into the project's `epoll` engine with echo, upload and download workloads; planning the architecture, measurement matrix and metrics (project phases 4–6).
4	ChatGPT (Niraj Kumar): chats "Explain Sockets And I O" and "Clarify Io Uring Project"	Understanding sockets, file descriptors and blocking versus non-blocking I/O; clarifying the `io_uring` design and the Phase 2 framework.
5	ChatGPT, Gemini (Chirag Agarwalla)	Building the benchmarking framework; reviewing the engines against the test bench; background on the theory and expected behaviour of the four mechanisms.
6	Claude Code with Claude Opus 5.5 (Nityanand Karmali): chat "Network I/O server benchmarking fixes"	Reviewing the code and first results against the TA feedback; fixing and extending the framework and experiments; analysing the final results; checking review comments on the engines; drafting this report and the presentation slides.

Relevant prompt queries and response `.md` files are shared on GitHub in the `AI_Usage` folder (https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO/tree/main/AI_Usage):

1) Kale Parth Hemant (`AI_Usage/select`):
   https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO/tree/main/AI_Usage/select
   Files: `claude_chat.md`, `gemini_chat.pdf`
2) Devvrat Hans (`AI_Usage/poll`):
   https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO/tree/main/AI_Usage/poll
   File: `ai_usage_poll.md`
3) Patil Nachiket Kiran (`AI_Usage/epoll`):
   https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO/tree/main/AI_Usage/epoll
   Claude chats: "Epoll echo server evolution part 1" and "Phase 4, 5, and 6 implementation part 2"
4) Niraj Kumar (`AI_Usage/io_uring`):
   https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO/tree/main/AI_Usage/io_uring
   ChatGPT chats: "Explain Sockets And I O" and "Clarify Io Uring Project"
5) Chirag Agarwalla (`AI_Usage/benchmark`):
   https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO/tree/main/AI_Usage/benchmark
   ChatGPT chats: "Build Linux IO Benchmarking Framework" and "Backend Testing Results"; Gemini chat: "Linux I/O Multiplexing Mechanisms Compared"
6) Nityanand Karmali (`AI_Used/Report_PPT`):
   https://github.com/devvrat-hans/CS331-T016-ScalableNetworkIO/blob/dev/nitya/AI_Used/Report_PPT/claude_chat.md
   Claude Code chat: "Network I/O server benchmarking fixes" (file: `claude_chat.md`)